GeoClaw-claude

User Guide v3.1.0

UrbanComp Lab · urbancomp.net · github.com/whuyao/GeoClaw_Claude · MIT License

适用读者	研究人员、城市规划师、GIS 从业者
内容概要	安装配置 / 自然语言分析 / Skill 开发 / 记忆系统 / Ollama 本地部署
前置要求	Python 3.9+，Linux / macOS / Windows，可选 Ollama（重集本地推断）
快速入门	pip install geoclaw-claude → geoclaw-claude init → geoclaw-claude ask '开始分析'

01 安装与初始配置

1.1 安装

```
# 正式安装（推荐）
pip install geoclaw-claude

# 开发者模式
git clone https://github.com/whuyao/GeoClaw_Claude
pip install -e geoclaw_claude_release/

# 可选扩展（栅格 / 路网 / Ollama）
pip install geoclaw-claude[raster]   # rasterio
pip install geoclaw-claude[mobility]  # trackintel
```

1.2 向导式初始化

```
geoclaw-claude init
# 交互选择 LLM Provider，输入 API Key
# 加载测试数据集验证安装
# 最终输出：GeoClaw 就绪！
```

1.3 手动配置

配置项	示例	说明
Anthropic	export ANTHROPIC_API_KEY=sk-ant-...	Claude Sonnet 4，达效最佳
Qwen	export QWEN_API_KEY=sk-... # DashScope	Qwen3-235B-A22B 旗舰山
Ollama	无需 Key，仅配置 IP+模型	100% 本地，隐私市场首选
OpenAI	export OPENAI_API_KEY=sk-...	gpt-4o-mini，全球通用
Gemini	export GEMINI_API_KEY=...	gemini-2.0-flash，免费额度可用

i 配置会自动保存到 ~/.geoclaw_claude/config.json，下次无需重输。多 Provider 同时配置时，按 anthropic →

gemini → openai → qwen → ollama 优先级自动选取。

02 快速上手

所有分析都可通过 自然语言命令 完成。下面按由浅入深给出三种典型工作方式。

2.1 CLI 命令行

```
# 单句分析
geoclaw-claude ask "读取 hospitals.geojson，生成 1km 缓冲区，统计覆盖居民点数"

# 多轮对话
geoclaw-claude chat  # 进入交互模式

# 运行内置 Skill
geoclaw-claude skill run hospital_coverage --radius_km 1.5
```

2.2 Python API

```
from geoclaw_claude.nl.agent import GeoAgent
from geoclaw_claude.config import Config

cfg = Config()  # 读取 ~/.geoclaw_claude/config.json
agent = GeoAgent(config=cfg)

# 自然语言对话
result = agent.chat("加载 hospitals.geojson")
result = agent.chat("生成 1km 缓冲区")
result = agent.chat("现在裁剪到 district.shp 范围内")

# 结束会话，写长期记忆复盘
agent.end("医院覆盖分析-2026-03")
```

2.3 直接调用分析函数

```
from geoclaw_claude.io.vector import load_vector
from geoclaw_claude.analysis.spatial_ops import buffer, clip
from geoclaw_claude.cartography.renderer import StaticMap

hospitals = load_vector('hospitals.geojson')
buf = buffer(hospitals, 1000)          # 1km 缓冲区
district = load_vector('district.geojson')
result = clip(buf, district)            # 裁剪到进研区域
StaticMap([result]).save('output.png')  # 出图
```

03 自然语言示例大全

以下按场景列出常用的中文指令示例，直接复制或稍加修改即可使用。

场景	示例指令
加载数据	加载 hospitals.geojson
缓冲区	对医院层生成 1.5 公里缓冲区
裁剪	将缓冲区裁剪到主城市局范围
叠置分析	求居民点与医院缓冲区的交集
核密度分析	计算上海市近三年交通事故热点，KDE bandwidth 设为 500
等时圈	以中心医院为起点，步行 15 分钟可达范围
最短路径	规划从 A 局到 B 医院的车行最短路径
分区统计	统计各街道 DEM 平均海拔和最大海拔
坐标转换	将图层从 GCJ-02 转换为 WGS-84
OSM 下载	下载北京市三环海淡的道路网络
制图输出	创建分级渲染主题地图并保存
交互地图	生成 Folium 可交互地图
记忆检索	查找上次分析医院覆盖率的结果
SRE 推理	请为武汉商业选址制定分析方案，识别最适方法

! 信心度 <0.55 时 GeoAgent 会先请求确认再执行。回复“是”或“yes”或直接修正该指令即可。

04 Ollama 本地部署

Ollama 使 GeoClaw 完全运行在本地，无需外部 API Key，适合数据隐私要求高的场景。

4.1 安装与启动

```
# 安装 Ollama (Linux/macOS)
curl -fsSL https://ollama.ai/install.sh | sh

# Windows: 下载 ollama-windows-amd64.exe 安装包

# 拉取模型 (选择任一)
ollama pull qwen3:8b        # 6GB, 默认推荐
ollama pull qwen3.5:9b     # 6GB, 类似 VRAM
ollama pull deepseek-r1:7b # 5GB, 推理尘强
ollama serve               # 启动服务, 默认 11434 端口
```

4.2 GeoClaw 配置

```
# 方式 1: 向导配置
geoclaw-claude config set llm_provider ollama
geoclaw-claude config set ollama_model qwen3:8b

# 方式 2: 直接 Python
from geoclaw_claude.config import Config
cfg = Config(llm_provider='ollama', ollama_model='qwen3:8b')

# 局域网其他机器部署
cfg = Config(llm_provider='ollama',
            ollama_base_url='http://192.168.1.100:11434/v1',
            ollama_model='qwen3.5:35b-a3b')
```

VRAM 选型指南

4-6 GB: qwen3:8b / qwen3.5:9b / deepseek-r1:7b / gemma3:9b
8-16 GB: qwen3:14b / llama4:scout / mistral-small3.1:24b
20 GB+: qwen3.5:35b-a3b | 40 GB+: deepseek-v3.1 / qwen3.5:122b-a10b

05 自定义 Skill 开发

Skill 是封装分析工作流的自描述模块，一次开发反复使用。

5.1 创建与注册

```
# my_skill.py
SKILL_META = {
    'name': 'flood_risk',
    'version': '1.0.0',
    'author': 'Your Name',
    'description': '洪水风险加权叠置分析',
    'tags': ['risk','raster'],
    'requires': ['geoclaw_claude >= 3.0.0'],
}

def run(ctx):
    buildings = ctx.get_layer('buildings')  # 获取已加载图层
    risk_raster = ctx.param('risk_raster')  # 参数
    threshold  = float(ctx.param('threshold', default=0.5))
    from geoclaw_claude.analysis.raster_ops import zonal_stats
    stats = zonal_stats(buildings, risk_raster, stats=['mean'])
    high_risk = buildings[stats['mean'] > threshold]
    comment = ctx.ask_ai(f'高风险建筑共 {len(high_risk)} 棋，请治理建议')
    return ctx.result(output=high_risk, commentary=comment)

# 安装注册
# geoclaw-claude skill install my_skill.py
# geoclaw-claude skill list
# geoclaw-claude skill run flood_risk --risk_raster dem.tif --threshold 0.6
```

> **i** SkillAuditor 安装时自动扫描安全风险（危险导入 / 未声明的网络请求 / 写入繁敏路径）。正常 Skill 预期 3-5 秒完成安装。

06 记忆系统实战

GeoClaw 拥有两级记忆。短期记忆自动管理；长期记忆帮助勿遇同样错误，积累分析经验。

6.1 长期记忆管理

```
from geoclaw_claude.memory.manager import MemoryManager
mm = MemoryManager()

# 手动存入重要发现
mm.save(title='医院分布不均',
         content='内圈覆盖超 90%，外圈缺口明显',
         tags=['医院','覆盖率'],
         importance=0.9)

# 关键词检索
results = mm.search('医院覆盖')
for r in results:
    print(r['title'], r['importance'])

# 语义检索（需 vector_search 初始化）
from geoclaw_claude.memory.vector_search import VectorSearch
vs = VectorSearch()
similar = vs.search('医院服务布局问题', top_k=3)
```

6.2 会话继续性

会话内支持代词引用，不必重复层名：

```
agent.chat("加载 hospitals.geojson")
agent.chat("生成 1km 缓冲区")
agent.chat("现在将它裁剪到 district.shp")   # “它” = 上一步缓冲区
agent.chat("在此基础上计算 KDE")            # “此基础上” = 裁剪结果
```

会话结束后应调用 agent.end() 自动写入复盘记忆

agent.end(title='上海商业分析-2026-03', auto_update_profile=True)

会写入：操作序列 / 执行统计 / AI 课 / 后续建议

07 SRE 地理推理引擎

SRE 帮助制定分析方案，识别 MAUP 风险，量化不确定性。正常工作流与 NL 对话无缝衔接。

7.1 快速使用

```
from geoclaw_claude.reasoning import reason_with_llm
from geoclaw_claude.nl.llm_provider import LLMProvider

llm = LLMProvider.from_config(cfg)
result = reason_with_llm(
    query="为武汉市神经外科诀断选址，识别最适空间分析方法",
    llm_provider=llm,
    datasets=['hospital_beds', 'population_density'],
)

print(result.reasoning_summary.summary_text())  # 推理摘要
for step in result.workflow_steps:              # 推荐执行步骤
    print(f'{step.order}. {step.description}')

# 不确定性评估 (Phase 3)
ua = result.uncertainty_assessment
if ua.overall_score > 0.7:
    print('警告: 高不确定性 ——', ua.recommendations)
```

7.2 对话内调用 SRE

```
agent.chat("请为下面分析设计工作流: 评估地鲁地带商业选址适宜性")
# GeoAgent 自动调用 SRE，返回推理过程 + 推荐方案
```

08 个性化配置

文件	路径	用途	编辑方式
soul.md	~/.geoclaw_claude/soul.md	系统行为边界（安全、等级、数据规则）	仅手动，4 章节锁定
user.md	~/.geoclaw_claude/user.md	用户专属领域 / 常用数据集 / 偏好风格	手动 + 对话自动更新

```
# user.md 示例
## Background
I'm a urban planner focusing on Wuhan city.
Familiar with Chinese coordinate systems (GCJ-02, BD-09).
"","## Preferred Datasets
- Medical facilities: hospitals.geojson (WGS-84)
"","## Analysis Preferences
- Buffer unit: meters
- Map style: publication
```

i GeoAgent 在对话中持续识别用户偏好， agent.end(auto_update_profile=True) 可批量写入 user.md。soul.md 中有四个安全锁定章节，对话无法修改。

09 常见问题

现象	原因 & 解决
请确认指令过多	GeoAgent 信心度<0.55 自动请确认。回复'是'确认，或更清晰重输指令
Ollama 连接失败	1) ollama serve 是否运行 2) 防火墙开放 11434 3) 检查 ollama_base_url
CRS 错误	输入数据精确指定坐标系：'将图层 CRS 设置为 EPSG:4326'或用 transform_layer 转换
内存不足	降低 cfg.ctx_max_tokens（建议 3000），或切换到更轻量 Ollama 模型
Skill 安装被拦截	SkillAuditor 检出危险调用。查看审计日志：~/.geoclaw_claude/logs/audit.log
地图无输出	StaticMap 需 matplotlib；无头服务器加 backend: import matplotlib; matplotlib.use('Agg')
记忆检索为空	查证长期记忆目录：~/.geoclaw_claude/memory/ 是否存在 .json 文件
中文误识别	llm_provider=ollama 时推荐 qwen3:8b 或 qwen3.5:9b，中文优化最佳